

```

"""
anomaly.py
-----
アノマリー検出レイヤー。
LLMが生成したテキストを3軸で測定し、集団からの乖離スコア（z候補）を算出する。

軸1: 語彙乖離    - TF-IDFベクトルのコサイン距離
軸2: 構造乖離    - 文長・句読点・繰り返しの統計的逸脱
軸3: 新語スコア  - 集団の語彙集合に存在しない語の割合

これらを合成してアノマリースコア [0, 1] を返す。
"""

import re
import math
import numpy as np
from collections import Counter
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_distances

class AnomalyDetector:
    def __init__(self):
        self.vocabulary_history: list[str] = []  # 過去の全テキストのpoolで語彙を構築

    # -----
    # メインAPI
    # -----

    def score_population(
        self, texts: list[str]
    ) -> list[float]:
        """
        テキストリスト全体を受け取り、各テキストのアノマリースコアを返す。
        スコアは [0, 1]。1に近いほどアノマリー。

        Parameters
        -----
        texts : list[str]
            各エージェントのLLM生成テキスト（エージェントと同じ順）

        Returns
        -----
        list[float]

```

各エージェントのアノマリースコア

```
"""
n = len(texts)
if n < 2:
    return [0.0] * n

scores_vocab = self._vocab_divergence(texts)
scores_struct = self._structural_divergence(texts)
scores_novelty = self._novelty_score(texts)

# 3軸を加重平均（それぞれ同重み）
combined = []
for i in range(n):
    s = (
        0.40 * scores_vocab[i]
        + 0.30 * scores_struct[i]
        + 0.30 * scores_novelty[i]
    )
    combined.append(float(np.clip(s, 0.0, 1.0)))

# 過去語彙プールを更新
self.vocabulary_history.extend(texts)
if len(self.vocabulary_history) > 500:
    self.vocabulary_history = self.vocabulary_history[-500:]

return combined

# -----
# 軸1: 語彙乖離 (TF-IDF コサイン距離)
# -----

def _vocab_divergence(self, texts: list[str]) -> list[float]:
    try:
        vec = TfidfVectorizer(
            min_df=1,
            stop_words=None,
            token_pattern=r"(?u)\b\w+\b",
            max_features=500,
        )
        # 過去語彙プールがあれば結合してfitする（集団語彙の安定化）
        fit_corpus = self.vocabulary_history[-200:] + texts if self.vocabulary_history else texts
        vec.fit(fit_corpus)
        matrix = vec.transform(texts).toarray()

        centroid = matrix.mean(axis=0, keepdims=True)
        # centroidが全ゼロ（全テキストが空等）の場合はスキップ
        if centroid.sum() == 0:
```

```

        return [0.0] * len(texts)

    distances = cosine_distances(matrix, centroid).flatten()
    # 正規化 [0,1]
    d_max = distances.max()
    if d_max == 0:
        return [0.0] * len(texts)
    return (distances / d_max).tolist()
except Exception:
    return [0.0] * len(texts)

# -----
# 軸2: 構造乖離 (文長・句読点・繰り返し)
# -----

def _structural_divergence(self, texts: list[str]) -> list[float]:
    features = []
    for t in texts:
        words = t.split()
        length = len(words)

        # 句読点密度
        punct_count = sum(1 for c in t if c in ".,:;!?!-...()[]\`'")
        punct_ratio = punct_count / max(len(t), 1)

        # 語の繰り返し率 (同じ語が何回出るか / 総語数)
        counts = Counter(w.lower() for w in words)
        repeat_ratio = (sum(v for v in counts.values() if v > 1) / max(length, 1))

        # 文の断片化 (句点で分割した文の数 / 語数)
        sents = re.split(r"[.!?]", t)
        frag_score = len([s for s in sents if s.strip()]) / max(length, 1)

        features.append([length, punct_ratio, repeat_ratio, frag_score])

    features = np.array(features, dtype=float)
    if features.shape[0] < 2:
        return [0.0] * len(texts)

    mean = features.mean(axis=0)
    std = features.std(axis=0) + 1e-9

    # 各エージェントの標準化距離 (マハラノビス的な簡易版)
    z_scores = np.abs((features - mean) / std).mean(axis=1)
    # sigmoid でスコア化
    scores = 1 / (1 + np.exp(-(z_scores - 1.0)))
    return scores.tolist()

```

```

# -----
# 軸3: 新語スコア（集団語彙に存在しない語の割合）
# -----

def _novelty_score(self, texts: list[str]) -> list[float]:
    if not self.vocabulary_history:
        return [0.0] * len(texts)

    # 過去の集団語彙
    pool_words = set()
    for t in self.vocabulary_history[-200:]:
        pool_words.update(w.lower() for w in t.split())

    scores = []
    for t in texts:
        words = [w.lower() for w in t.split()]
        if not words:
            scores.append(0.0)
            continue
        novel = sum(1 for w in words if w not in pool_words)
        scores.append(novel / len(words))

    return scores

# -----
# デバッグ・観測用
# -----

def explain(self, text: str, texts: list[str]) -> dict:
    """単一テキストの各軸スコアを返す（観測・ログ用）。"""
    idx = texts.index(text) if text in texts else 0
    v = self._vocab_divergence(texts)
    s = self._structural_divergence(texts)
    n = self._novelty_score(texts)
    return {
        "vocab_divergence": round(v[idx], 4),
        "structural_diverge": round(s[idx], 4),
        "novelty_score": round(n[idx], 4),
        "combined": round(0.4 * v[idx] + 0.3 * s[idx] + 0.3 * n[idx], 4)
    }

```